

Lab 02 - Tokenize business tickets

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Use a real Hugging Face tokenizer and explain IDs, masks, subwords and unknown terms.

Alignment: K3, K6, K7; A1. Scheduled hands-on time: 45 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- tickets.csv
- vocab.txt
- requirements.txt

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Create a virtual environment: `python3 -m venv .venv`. Activate with `source .venv/bin/activate` on macOS/Linux, or `.venv\Scripts\activate` on Windows.
2. Install dependencies with `python -m pip install -r requirements.txt`. This lab uses a local vocabulary and downloads no pretrained model weights.
3. Run `python run.py`. Inspect `output.json` for token strings, `input_ids` and `attention_mask`. Match the text "the brown fox jumps over the lazy dog" to its token IDs.
4. Locate the unknown-word example. Explain why [UNK] appears, then compare the vocabulary entries `deliver` and `##y` with the tokenizer output for `delivery`.
5. Change `max_length` in `run.py` from 24 to 8 and rerun. Record which business information is truncated. Restore 24 and rerun for your final evidence.
6. Use `PROMPTS.md` to ask for an explanation grounded only in the actual output. Save a short explanation of special tokens, masking and truncation in `learner-output.json`.

Acceptance checks

- Real Hugging Face `BertTokenizerFast` output exists.
- Token IDs and masks have equal length for every example.
- The learner explains unknown tokens, subword boundaries and truncation.

Run `python verify.py` after `run.py` (use `python3` if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's `requirements.txt`.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.

- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.